

TSFS12 HAND-IN EXERCISE 5

Learning for Autonomous Vehicles

October 4, 2026

1 OBJECTIVE

The objective of this exercise is to introduce different types of learning methods relevant for autonomous vehicles. A first task is to explore reinforcement learning in a scenario with a grid world with a moving autonomous vehicle, modeled as a particle, driving in a world with inherent uncertainties modeled using probabilities. A second task is to investigate neural networks as a means of making trajectory predictions of agents in a dense traffic scenes. Compared to the previous four hand-in exercises in this course, most of the implementation code required is provided in this exercise. The intention of the exercises is to provide an understanding and hands-on experience with the methods discussed during the lectures on learning for autonomous vehicles. In the extra assignment for higher grades in Appendix A, neural networks and reinforcement learning are combined to solve a more complex highway-driving scenario with multiple vehicles in several driving lanes.

2 PREPARATIONS BEFORE THE EXERCISE

Before performing this hand-in exercise, please make sure that you have read and understood the material covered in:

- Lectures 10–11.
- Sections 4.1–4.4 and 6.1–6.5 in Sutton, R. S., & A. G. Barto: *Reinforcement learning: An introduction*. MIT Press, 2018, [4].

To get code skeletons to be used for the different exercises, download the latest files from <https://gitlab.liu.se/vehsys/tsfs12> and familiarize yourself with the provided code. The data needed for the exercise on trajectory prediction using neural networks is available directly available in the student labs and you do not need to download the data. The data can be found at `/courses/tsfs12/trajectory_prediction`. If you are working on your own computer the data is available in Lisam under **Course documents/hi5_trajectorydata.zip**. Please download the file and unzip it in the same directory as the remaining files. The uncompressed data occupies about 1.3 GB of disk space.

For the value-iteration and Q-learning exercises, you are free to choose language for implementation and code skeletons are available in both Matlab and Python. For the neural-network assignment and the deep Q-learning assignment for higher grades, only Python is available. Note, though, that you will only be required to write very limited lines of code in these exercises, so a background in Python is not necessary. In the student labs, all packages needed are pre-installed in the virtual environment activated by

```
1 source /courses/tsfs12/env_hi5/bin/activate
```

Important note: The name of the environment to use is specific for this exercise and is different from the one used in the previous exercises. Please make sure to use the correct environment when running the code.

3 REQUIREMENTS AND IMPLEMENTATION

The objective of the exercise is to get initial practical understanding of different methods for learning for autonomous vehicles and evaluate their properties in some example scenarios. Therefore, to pass this exercise you should solve the following tasks:

- Apply value iteration to solve a Markov decision process in a grid-world scenario under known environment dynamics.
- Apply Q-learning to the same problem as in the previous task, but without *a priori* known state-transition probabilities.
- Evaluate trajectory prediction using neural network models.

The exercise is examined by submitting the following on the Lisam course page.

1. Runnable code. If your code consists of several files, submit a zip-archive with your files. Do not include files for the language that you did not use.
2. A report that
 - that answers the asked questions, including relevant plots, and other questions that you have encountered during solving this exercise
 - important to include your reflections in the document, and not just report results and code. It is necessary to include a discussion whenever requested in the questions.
 - includes a concluding discussion

The report is expected to be a neat, self-contained, document in PDF format. You are encouraged, but not required, to use the templates provided at the course git-repo.

It is allowed to discuss the exercises on a general level with other course participants. However, code sharing outside groups is not allowed. Moreover, both students in the group should be fully involved in performing all exercises, and thereby be prepared to answer questions on all subtasks.

See Appendix A for the extra assignment needed for higher grades. The extra assignment is done individually and is submitted by a document answering the questions, including suitable figures, plots, and tables. The document is to be submitted in PDF format. The code should also be submitted as a zip-archive. There is only pass/fail on the extra exercise and the document can not be revised or extended after first submission. It is not required to get everything correct to pass the assignment, we assess the whole submission. You are of course welcome to ask us if you have questions or want us to clarify the exercise.

Use of AI Tools

AI tools (ChatGPT, Claude, Copilot, etc.) are allowed for this course, including hand-in exercises. Use them to get unstuck, explore ideas, debug code, or improve the language and clarity of your report, but the report itself must be written by you, not generated by AI.

Rules:

- **Allowed:** using AI to proofread, rephrase, restructure, or polish text you’ve written; getting help with code, debugging, or understanding concepts.
- **Not allowed:** having AI generate report content, such as sections, paragraphs, or explanations, that you then submit as your own writing.
- Every submission must include a short statement describing how AI tools were used (or “*No AI tools were used,*” if applicable). Be as specific as you can, for example “*Used Claude to debug a recursion error in Task 2*” or “*Used ChatGPT to improve phrasing in Section 3.*”. A general statement like “*AI was used for coding help*” is not sufficient.
- You must be able to explain and defend every part of your submission, regardless of how it was produced.
- Verify AI output before submitting since it can be confidently wrong.
- Any exercise where AI use is restricted further will say so explicitly.

Submissions without an AI usage statement will be treated as incomplete.

Since these exercises examine this course, we would like to ask you not to distribute or make your solutions publicly available, e.g., by putting them on a public GitHub repository. A private repository on, e.g., gitlab@liu is of course fine.

Thanks for your assistance!

This hand-in exercise is divided into two different parts, with subtasks to be performed for each. The required implementation code for solving the different tasks is to a large extent provided for the exercises in the skeleton files.

4 REINFORCEMENT LEARNING USING VALUE ITERATION AND Q-LEARNING

In this part of the hand-in exercise, we will investigate two different algorithms for solving a Markov decision process using reinforcement learning. More specifically, we will investigate value iteration under known environment dynamics as well as Q-learning in the more complex case of *a priori* unknown state-transition probabilities. The assignment is based on

- Example 6.6 in Sutton, R. S., & A. G. Barto: *Reinforcement learning: An introduction*. MIT Press, 2018.

The scenario is an autonomous vehicle, here modeled as a particle, moving in a grid world from state to state (see Figure 1). In each state in the grid world, the vehicle has four different actions to choose from: left, right, up, and down. These are counted as number 1–4 in Matlab and 0–3 in Python in the implementation, in the respective order. Because of disturbances such as wind acting in the modeled world, applying a certain action does not necessarily mean that the vehicle ends up with moving in the intended direction; instead there is a certain probability that the vehicle will move sideways compared to the intended motion. If the vehicle applies an action that would bring it outside of the world, it remains in the same state and once again receives the reward associated with that state. The overall task is to compute a policy (i.e., actions as function of state) such that it maximizes the expected accumulated reward for the vehicle from start to the goal state.

The motion starts in the lower left corner (S) and the goal is to move to the lower right corner (G). There are certain states in the world that are occupied by obstacles, and

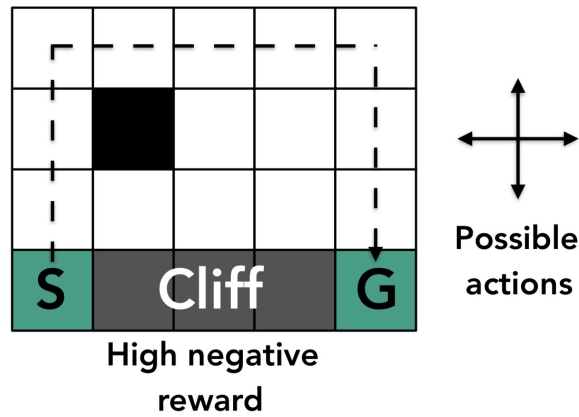


Figure 1: The grid world where the autonomous vehicle is moving from start (S) to goal (G).

the states between the start and goal state in the lower row comprise a cliff where the vehicle gets stuck. The cliff states and the goal states are here considered as terminal states.

The reward associated with the cliff states is selected as a large negative number. The reward for the goal state is chosen to be zero and the remaining free states (i.e., states not occupied by obstacles) have a small negative reward associated with them. The latter choice is to make it cost to take (unnecessary) steps in the world before reaching the desired goal state, which can be interpreted as trying to keep the path length as short as possible, though under consideration of the risk of getting stuck in the cliff states.

The code required to perform the exercises in the following is available in a number of provided files. In Matlab, the main files are `rl_val_iter.m` and `rl_q_learning.m`. Auxiliary functions are available in the file `select_eps_greedy.m` as well as in the directory `Functions`. In Python, there are notebooks with corresponding content called `rl_val_iter.ipynb` and `rl_q_learning.ipynb`. In the Python version, the auxiliary functions for this task are available in the file `grid_world.py`. Note that the functions `next_state` and `select_eps_greedy` are defined directly in the notebook `rl_q_learning.ipynb`. There are also script versions available if you prefer in the files `rl_val_iter.py` and `rl_q_learning.py`. The parameters defining the scenario are stored in the struct/dictionary `params`, and the properties of the parameters are described in the commented code.

4.0.1 Value Iteration

Exercise 4.1. Carefully read through the code provided in the file `rl_val_iter.{m, ipynb, py}` and the associated auxiliary files, and study in particular the variables available in the struct/dictionary `params` that define the Markov decision process to be solved.

Exercise 4.2. In the first part of the exercise, we will investigate value iteration for determining the optimal value function $V(s)$ and the corresponding optimal policy $\Pi(s)$. Explain mathematically, and in your own words, what the output of the value function corresponds to.

Exercise 4.3. Study the algorithm for value iteration in Lectures 10–11. Consider the computation of an updated value for $V(s)$ on Line 6 in the algorithm. The function `p_grid_world` (available in the directory `Functions` in Matlab and in the file `grid_world.py` in Python) implements important computations needed for perform-

ing this update. Study this function. Explain what the outputs from this function correspond to in terms of the grid world and the scenario considered.

Exercise 4.4. Execute the main loop of the value iteration in the file `rl_val_iter.{m, ipynb, py}` for the default values of the parameters. For each iteration, the current value function and the associated actions are shown in two different plots for each state in the grid world (using the function `plot_value_and_policy`). The execution is paused between each iteration, and the execution is continued by pressing enter on the keyboard. Run the iterations until convergence.

Exercise 4.5. Which is the optimal path from start to goal? Note that the solution does not only provide this path, but an optimal policy for all states. Is the optimal policy obtained in agreement with your intuition? Relate the numbers in the plot illustrating the value function $V(s)$ to your interpretation of that function from Exercise 4.2.

Exercise 4.6. Now we will investigate how changes in the state-transition probabilities affect the resulting optimal policy. Vary the probability of the car to be subject to disturbances (the parameter `P_move_action`, in the interval $(0,1]$). What happens if you set this parameter to 1 and what does it mean in practice for this scenario? Is the result in agreement with your intuition on how to optimally negotiate the uncertainty and associated risks in the scenario at hand?

Exercise 4.7. Vary the discount factor (the parameter `gamma`) and study the results on the optimal policy. Do you see any noticeable effects, and are the results in agreement with your intuition?

Exercise 4.8. Finally, vary the negative reward received for ending up in the cliff states (the parameter `R_sink`). How does it influence the optimal path from the start state to the goal state, if it is made significantly less or more negative compared to the negative reward received for the remaining obstacle-free states?

4.0.2 Q-Learning

Now we will investigate the case when the state-transition probabilities are *a priori* unknown. The same scenario and Markov decision process as in the previous exercise will be solved, but now using Q-learning instead. The learning rate is determined by the parameter `alpha` and a total number of `nbr_iters` episodes of Q-learning are performed. The trade-off between *exploration* of new actions and states and *exploitation* of already acquired information during the iterations is decided using the epsilon-greedy strategy, where `eps` is the parameter governing the trade-off. This strategy is implemented in the function `select_eps_greedy`.

Exercise 4.9. Read through the code provided in the main implementation file `rl_q_learning.{m, ipynb, py}` and the associated auxiliary files. In particular, study the variables available in the struct/dictionary `params`. Q-learning relies on iteratively updating the function Q by running a sequence of episodes where the agent interacts with the environment. This interaction could be either in simulation or in experiments, here we will use simulations in the grid world.

Exercise 4.10. Study the algorithm for Q-learning from Lectures 10–11. A key part is to choose an action, making a state transition, and receiving an associated reward. This procedure is implemented in the function `next_state` (available in the directory `Functions` in Matlab and in the notebook `rl_q_learning.ipynb` in Python). The Q function is subsequently updated using the temporal-difference (TD) error (Line 8 in the algorithm). Explain in your own words what the output of the

Q -function corresponds to. From this function, the optimal policy $\Pi(s)$ could also be extracted. Describe how this could be made.

Exercise 4.11. Execute the main loop of the Q-learning (`rl_q_learning.m` in Matlab and `rl_q_learning.ipynb` in Python) for the default values of the parameters (we here start with the case where there are no disturbances and the vehicle thus always moves according to the specified action, i.e., `P_move_action` = 1.0). In contrast to the value iteration, only the final resulting value function and corresponding actions are shown in the plots (using the function `plot_value_and_policy`) since there are typically many more iterations required for Q-learning with unknown state-transition probabilities. In addition, a plot is shown where the averaged accumulated rewards for the episodes are visualized. Does the Q-learning converge to the same optimal policy as the value iteration (if you have the same parameters for the Markov decision process)?

Exercise 4.12. Vary the probability of the car to be subject to disturbances (i.e., the parameter `P_move_action`) to something slightly lower than 1. How does it affect the Q-learning? Does the Q-learning find the same optimal policy from start to goal as the value iteration (if you have the same parameters for the Markov decision process)? How about the policy at the states other than the start state?

Exercise 4.13. An important parameter in the Q-learning is the parameter `eps` governing the exploration vs. exploitation strategy. Typically, it is desirable with more exploration in the beginning of the episodes when less information has been acquired about the system, and then in the end the agent relies almost completely on the information that has already been acquired.

Update the implementation such that the parameter `eps` is made dependent on the episode number, and successively decreases towards zero, starting from one, when the iteration number increases. The iteration number `itr_nbr` is available as an input to the function `select_eps_greedy`. Do you see any improvements in the performance of the Q-learning and its convergence to the optimal policy? Study also the accumulated reward plot. As a further extension, several exploration/exploitation phases could also be performed, i.e., the parameter `eps` starts at 1 and decreases towards zero and then after a pre-defined number of episodes it is increased again to 1 to re-start the exploration and then subsequently decreased to zero again. Implement also such a strategy. Compare the plots of the accumulated reward as function of episode for the different strategies. Does this more advanced strategy improve performance in terms of the average accumulated reward towards the end of the learning (when `eps` is close to zero again)?

Exercise 4.14. Discuss what is the main difference between value iteration and Q-learning. Why does value iteration converge in much fewer iterations than Q-learning?

5 CONTEXT-AWARE TRAJECTORY PREDICTION USING NEURAL NETWORKS

When doing automated planning and control for an autonomous vehicle, it would be highly beneficial if the future trajectories of the surrounding vehicles would be known. Then, safe and proactive plans could be made.

Consider for example the roundabout traffic scene in Figure 2 where predictions of vehicles during a 5-second horizon is shown. The solid “tails” show a short history of the vehicle’s trajectory and is used as input to the prediction algorithm. The dotted lines then show the predicted future trajectory. These predictions are of course uncertain, but they do give valuable information for an autonomous agent. Developing physical, or rule-based, models for future driver behavior is very difficult, and therefore data-driven techniques, e.g., based on neural networks is an interesting approach.

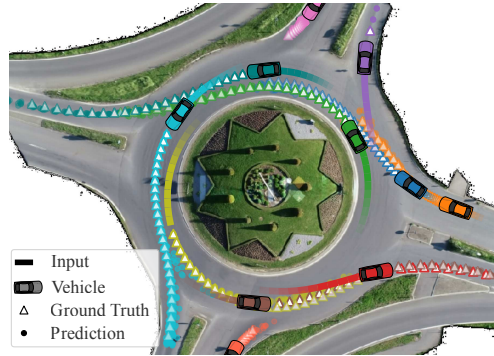


Figure 2: Example of a traffic scene with trajectory predictions. Figure from [6].

The objective of this exercise is not to develop or train such models, but to making small experiment with pre-trained models to get an initial understanding of possible challenges and limitations. To be useful in this course, the models used in the exercise are somewhat simplified and do not have state-of-the-art performance. However, the models share architectural components and properties with more advanced models. If you are interested in this topic and want to know more, see for example [6].

5.1 The problem setup and dataset



Figure 3: Traffic scenarios where the SIND dataset was collected.

This exercise utilizes data from the SIND dataset [7] to train and evaluate a deep neural network model for motion prediction. The dataset consists of several scenarios with various agents, including cars, bikes, and pedestrians, moving around in different signalized intersections. See Figure 3 for birds-eye-view of the traffic scenes used in the dataset. For this exercise, the data has been pre-processed and test-data has been generated using the Dronalize toolbox¹.

This exercise relies heavily on Python and PyTorch libraries. For the exercises you are not expected to have any prior knowledge of either Python or PyTorch. However, you are encouraged to explore the documentation of these libraries to gain a deeper understanding of the code if you have the required prior knowledge. You will be provided with a set of pre-trained models, and your task is to evaluate and experiment with the models on a test set and analyze the results. You will be able to run all code

¹ Dronalize: <https://github.com/westny/dronalize>

in a Jupyter notebook and everything needed to run the exercise is pre-installed in the student computer labs.

5.1.1 Short primer for running the exercise

The code for this exercise is provided in a Jupyter notebook, and you can run it as any of the other exercises. The notebook is located in the `python/trajectory_prediction` subfolder of the hand-in folder.

To run, start a terminal and change directory to the hand-in folder. Then, copy the commands below (except the `%`) to activate the python environment and start jupyter lab:

```
1 % source /courses/tsfs12/env/bin/activate
2 % jupyter lab
```

I also recommend to watch the hand-in introductory video to get a better understanding of the exercise. You can also of course use another IDE (VSCode, Pycharm) that can run jupyter notebooks.

5.2 Data and Models

The data and models are described in detail in the provided notebook `python/trajectory_prediction/trajectory_prediction.ipynb`.

The models used in the exercise are:

- A simple model with a constant velocity assumption
- An attention-based neural network model with no usage of a map
- Attention-based model with a feedforward neural net map encoder
- Attention-based model with a Graph Attention neural network (GAT) map encoder

If you have historical data for all agents in a traffic scene in a variable `data`, you can use the following code to get predictions from the model:

```
1 _, pred, attn, _ = model(data)
```

When using the model for prediction, four output arguments are returned. However, here only the 2nd and 3rd output arguments are used. The 1st and 4th are discarded using the `_` character. The `pred` variable contains the predicted future positions of all agents in the scene, while the `attn` variable contains the attention weights used by the model. More details on the data, the prediction, and the attention weights will be explored in the exercises next.

See the notebook for more details on the models and how to use them.

5.3 Exercises

The goal is to predict the future positions of the agents based on their past positions and velocities. Start by reading the provided notebook carefully and run the code to get familiar with the data and models.

The exercises are divided into 4 sections

- Data exploration
- Inspection of the models and some initial predictions
- Explore the attention mechanism

- Explore and evaluate the different models

For each section there is a list of tasks to be performed. The notebook runs as provided, but you are expected to extend with your own investigations and comments.

A EXTRA ASSIGNMENT

Deep Q-Learning for Highway Driving

In this exercise, you will explore a reinforcement-learning approach, Deep Q-Learning [3], to navigate in the simplified highway scenario illustrated in Figure 4. The objective

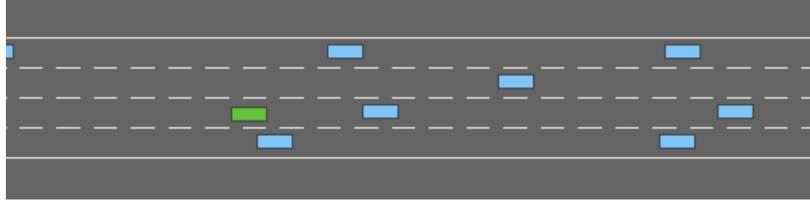


Figure 4: A snapshot from the highway scenario where the green vehicle is the ego vehicle and the blue vehicles are surrounding traffic.

of the exercise is to explore a model-free approach to reinforcement learning in a non-trivial task. The exercise will require very limited coding, and mainly aims at exploring and understanding existing code and models. There is pre-prepared code for all exercises. For more background on the approach, see the slides from Lectures 10–11, the reading instructions in Section 2, and the paper [5] (also [3] can be read for reference). The assignment is examined by submitting any relevant code you may have written and a short report with relevant plots and discussions.

The Highway Scenario — Actions, Observations, and Rewards

The exercise is based on the highway environment [1] for Gymnasium². The objective is to navigate fast and smoothly on the highway without any collisions using observations of relative positions and velocities of the surrounding traffic. The scenario will end if there is a collision *or* a maximum of 40 actions have been taken.

In the scenario setup used here, there are 5 discrete **actions** available:

$$\mathcal{A} = \{\text{lane_left}, \text{idle}, \text{lane_right}, \text{faster}, \text{slower}\}. \quad (1)$$

An **observation** is a 5-by-5 matrix where each row corresponds to a vehicle and the columns correspond to the features (presence, x, y, v_x, v_y). The first row corresponds to absolute positions and velocities of the ego vehicle, and the four remaining rows correspond to four surrounding vehicles and their relative positions and velocities with respect to the ego vehicle (normalized). A sample state/observation can look like

$$s = \begin{pmatrix} 1 & 1 & 0.5 & 0.416 & 0 \\ 1 & 0.133 & -0.25 & -0.019 & 0 \\ 1 & 0.277 & 0.25 & -0.030 & 0 \\ 1 & 0.427 & 0.25 & -0.052 & 0 \\ 1 & 0.583 & -0.25 & -0.020 & 0 \end{pmatrix} \quad (2)$$

which means that, e.g., the surrounding vehicle 1, corresponding to the second row, has relative position and velocity

$$(\Delta x, \Delta y, \Delta v_x, \Delta v_y) = (0.133, -0.25, -0.019, 0).$$

Positions are normalized and a y value of 0 means center of the leftmost lane, 0.25 center of the the second leftmost, etc., until $y = 0.75$ for center of the rightmost lane.

The **reward** r is upper bounded by 1 and positive rewards are given for higher speed and driving in the rightmost lane, while collisions have a negative reward.

² <https://gymnasium.farama.org/>, former Open AI Gym

Exercises

Skeleton files are available in `extra.{ipynb/py}` and the Deep Q-learning agent is implemented in the file `dqn_agent.py`. The implementation is based on the DQN agent in [2] but is simplified for educational purposes.

Exercise A.1. In Section 4, Q-learning was implemented by direct tabulation of the state-action value-function $Q(s, a)$ where s is the state and a the action. This was possible because of a discrete and finite state space of limited size, which is not the case here since observation/state $s \in \mathbb{R}^{5 \times 5}$ is a continuous quantity.

One approach would be to discretize the state space. To illustrate why this approach is difficult here, assume that the 20 features (here ignore the 5 presence features) are discretized into 3 discrete values, e.g., {low, medium, high}. What is then the size of the Q-table?

Exercise A.2. Based on the observations in the previous exercise, one approach to handle the complexity is to make a continuous functional approximation of the Q-table with a deep feed-forward network $Q : \mathbb{R}^{5 \times 5} \rightarrow \mathbb{R}^5$. Note the slight difference here, instead of a function from s and a to a scalar Q-value, the network function is from a state $s \in \mathbb{R}^{5 \times 5}$ to $\mathbb{R}^5$, the Q-values for all 5 actions.

The architecture for the provided pre-trained model is shown in Figure 5. The model has 3 dense layers with ReLU activation functions and 256, 256, and 5 units in the respective layer. This results in a model with 73 753 parameters. Comment on the number of parameters in the model and relate to your answer in Exercise A.1.

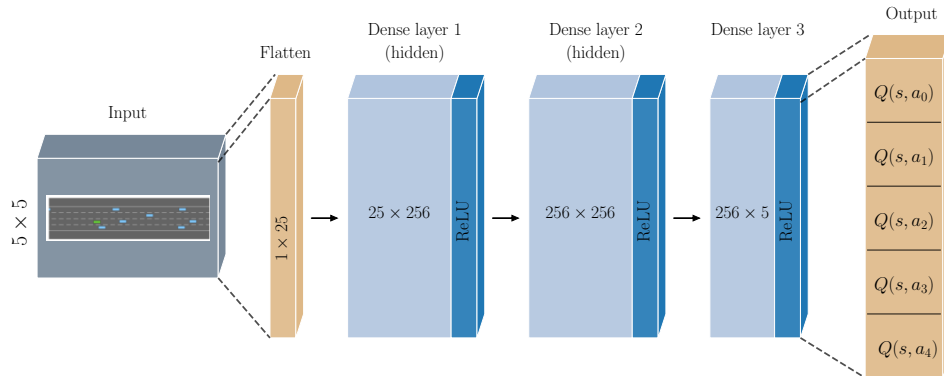


Figure 5: The neural network functional approximation of $Q(s, a)$.

Exercise A.3. The Deep Q-learning agent is fully implemented in the file `dqn_agent.py`. Study the class methods `train` and `update` and write down in pseudo code how the training loop works. In particular, note how there is not one but two neural networks interacting during learning. See the first page of [3] or the background in [5] for a description of the two networks and how they interact. Train the model for a few episodes, with scene rendering turned on, to further understand the process.

Training a well-functioning agent for this problem takes quite a long time, at least 4-5 000 episodes are necessary. Therefore, a pre-trained model is provided that has been trained for 8 000 episodes that you can use if you like. Figure 6 shows some statistics from a training process with multiple exploration phases. The cumulative reward is the total reward obtained during an episode and 40 is the maximal value since $r \leq 1$ and the episode stops at collision or 40 steps. The exploration/exploitation plots show the probability of taking a random action vs. using the current learned policy, the

ε -greedy approach. Comment on the plots, in particular how the cumulative rewards should be interpreted with respect to the exploration/exploitation profile.

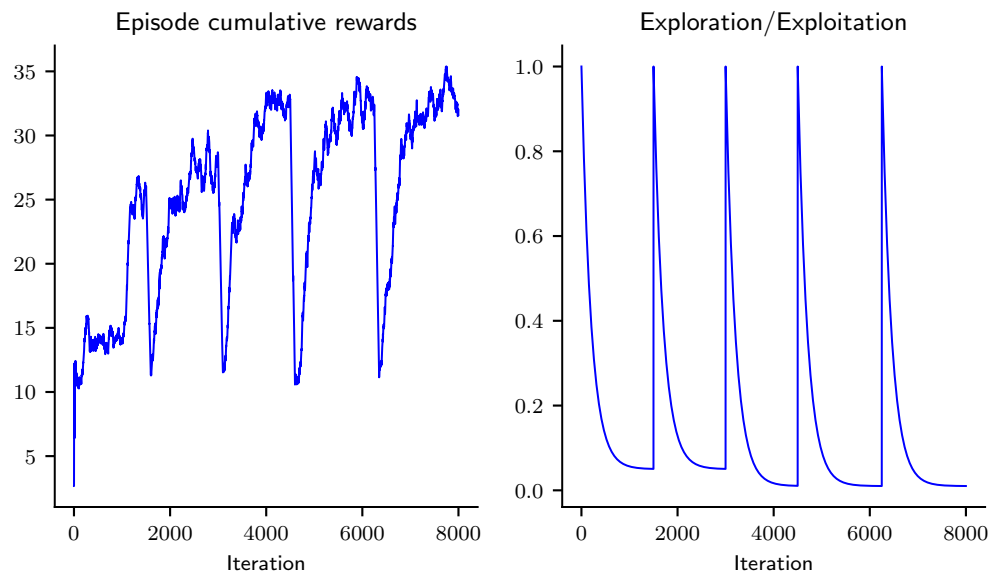


Figure 6: Cumulative reward and exploration/exploitation probability during training. The plots are averaged over 100 episodes.

Exercise A.4. This exercise will explore the pre-trained model and the obtained function $Q(s)$. First, generate an observation s from a random starting state and render the scene by executing

```
1 s = env.reset()
2 env.render()
```

Print the observation s and evaluate the pre-trained $Q(s)$. Interpret the results and explain which action that will be taken. The actions are ordered as in (1). Compare with the model from Exercise A.2 that has only been trained for a few episodes.

To understand the Q -function, design an observation by hand, for example one where there is no vehicle in front or an example with a vehicle in front but vacant space in neighboring lanes. Evaluate this particular observation on the pre-trained model, does the model act the way you expect?

Now, run the pre-trained model on a number of random scenarios (not with the observation designed above) to get further understanding of how it performs. Increase the traffic density (see code) to increase the difficulty.

Exercise A.5. Until now, the state-action value-function $Q(s, a)$ has been explored and now we will investigate the value function $V(s)$ closer. In the scenario, the reward satisfies $r \leq 1$. Find an upper bound on $V(s)$ and $Q(s, a)$ as a function of the discount factor γ . What is this bound in this case, with a discount factor $\gamma = 0.8$?

Run a scenario with the pre-trained model and compute $V(s)$ for all visited states in the simulation. Plot the result and compare with the bound. The value $V(s)$ can be computed using the method available in the `Q` class.

Try different scenarios and select one that you find interesting. Comment on how the value of $V(s)$ can be interpreted.

Exercise A.6. This problem has to be considered rather simple and idealized compared to a real-world situation. Make a short reflection on the role of reinforcement-learning algorithms for autonomous vehicles, pointing out possibilities, challenges, and potential problems. Comment on ideas how to overcome possible issues.

REFERENCES

- [1] Edouard Leurent. An environment for autonomous driving decision-making. <https://github.com/eleurent/highway-env>, 2018.
- [2] Edouard Leurent. rl-agents: Implementations of reinforcement learning algorithms. <https://github.com/eleurent/rl-agents>, 2018.
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236).
- [4] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT Press, 2018. <http://incompleteideas.net/book/the-book-2nd.html>.
- [5] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016. <https://arxiv.org/abs/1509.06461v3>.
- [6] Theodor Westny, Joel Oskarsson, Björn Olofsson, and Erik Frisk. MTP-GO: Graph-based probabilistic multi-agent trajectory prediction with neural odes. *IEEE Transactions on Intelligent Vehicles*, 8(9):4223–4236, 2023. <https://arxiv.org/abs/2302.00735>.
- [7] Yanchao Xu, Wenbo Shao, Jun Li, Kai Yang, Weida Wang, Hua Huang, Chen Lv, and Hong Wang. SIND: A drone dataset at signalized intersection in china, 2022. <https://arxiv.org/abs/2209.02297>.